

COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI ICADE CIHS

Sprint 4

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

Índice **2**
Introducción **3**
Objetivo del Escenario 4 3
Alcance del Sprint 4 3
Metodología **3**
1. Despliegue del clúster EMR mediante CloudFormation 4
2. Acceso al nodo primario del clúster EMR 4
3. Ejecución de un script PySpark con spark-submit 4
Implementación del Algoritmo de Montecarlo 4
Descripción del Código 5
Configuración de Pruebas 6
Herramientas de Monitorización 7
Resultados **15**
Datos Experimentales 15
Análisis de Rendimiento 16
Conclusión **21**
Comparativa Escenario 3 - Escenario 4 21
Reflexión y conclusión final 22

Introducción

Este informe documenta el desarrollo y análisis del cuarto y último escenario del proyecto BENCHDATA, correspondiente a la evaluación de **Hadoop/Spark** con cargas computacionales significativamente superiores a las utilizadas en los escenarios previos, en concreto multiplicadas por 10.

Objetivo del Escenario 4

El Escenario 4 tiene como propósito evaluar el comportamiento y escalabilidad Amazon Elastic MapReduce (EMR) utilizando infraestructura como código (IaC) cuando se enfrenta a volúmenes de datos masivos: **50M, 100M y 500M de puntos**. Este sprint nos va a permitir:

- **Validar la escalabilidad real** de Spark en el límite superior de la capacidad de la infraestructura.
- **Identificar si se ven o no algunos cuellos de botella**, como nos pasaba en los sprints anteriores.
- **Comparar directamente** con los resultados del Sprint 3 para entender el impacto del escalado 10x y del nuevo método de ejecución.

Alcance del Sprint 4

Durante este cuarto sprint, se han llevado a cabo las siguientes actividades:

- Ejecución del algoritmo de Montecarlo con cargas 10x superiores: **50M, 100M, 500M puntos.** Aunque mantenemos el mismo código y solo cambiamos esas cifras.
- Registro de métricas de rendimiento: tiempo, velocidad (throughput), estimación de π
- Análisis comparativo E3-E4 para evaluar el comportamiento del escalado.

Con este sprint completamos la evaluación y exploración de Hadoop/Spark, tratando de entender mejor su comportamiento en escenarios que se aproximan a casos de uso reales de Big Data.

Metodología

A continuación se describe, de forma ordenada y comprensible, todo el proceso seguido para desplegar un clúster de Amazon EMR, conectarse al nodo primario y ejecutar un script the python utilizando `spark-submit`.

1. Despliegue del clúster EMR mediante CloudFormation

Siguiendo las instrucciones del documento *Despliegue Amazon EMR*, creamos un clúster EMR utilizando infraestructura como código (IaC) a través del servicio **AWS CloudFormation**.

El proceso ha consistido en:

1. Acceder a la consola de AWS (en la región de España)
2. Abrir el servicio **CloudFormation**.
3. Seleccionar **Create Stack** y cargar la plantilla proporcionada (template IaC).
4. Rellenar los parámetros necesarios: nombre del stack, VPC, subred, tipo de instancia, etc.
5. Confirmar capacidades (IAM) y lanzar la creación del stack.

Una vez completado este proceso, nos apareció un nuevo clúster activo en la consola de Amazon EMR.

2. Acceso al nodo primario del clúster EMR

Nos contactamos mediante conexión SSH al **nodo primario**, donde ya podemos ejecutar scripts de python y tareas distribuidas, dentro del propio clúster. Y ya podemos reutilizar el código del anterior sprint.

3. Ejecución de un script PySpark con `spark-submit`

Una vez dentro del nodo primario, ejeuctamos nuestro script `escenario4.py` utilizando el comando :

```
spark-submit --master yarn --deploy-mode client escenario4.py
```

Implementación del Algoritmo de Montecarlo

Fundamento Matemático: El método de Montecarlo para calcular π se basa en la relación geométrica entre un cuadrado y un círculo inscrito en él, manteniendo la misma base matemática que los escenarios anteriores.

Al generar puntos aleatorios uniformemente distribuidos en un cuadrado de lado 1 y contar cuántos caen dentro del círculo de radio 1, podemos estimar π mediante la fórmula:

$$\pi \approx 4 \times \frac{\text{puntos dentro del círculo}}{\text{puntos totales}}$$

Este método es ideal para evaluación de sistemas distribuidos porque:

- Es altamente paralelizable (cada iteración es independiente)
- La carga computacional es ajustable (variando el número de puntos)
- Proporciona una métrica clara de precisión (comparando con el valor real de π)
- Permite evaluar la eficiencia del modelo de programación distribuida de Spark (transformaciones y acciones sobre RDDs)

Descripción del Código

El script implementado en Jupyter Notebook, que contiene el código que adjuntamos en el script de `escenario4.py` es idéntico al del sprint anterior pero cambiando la cantidad de puntos en 'Ejecución paralela con RDDs'.

Configuración de la conexión y del contexto Spark:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \

      .appName("MonteCarloPi") \

      .master("spark://7fd044fca667:7077") \

      .getOrCreate()

sc = spark.sparkContext
```

Función de cálculo distribuida:

```
import random

def montecarlo_pi_in_circle(puntos):
```

```
    rnd = random.random
    count = 0
    for _ in range(puntos):
          x = rnd()
          y = rnd()
          if x*x + y*y <= 1:
               count += 1
     return count
```

Ejecución paralela con RDDs:

```
NUM_TAREAS = 200
nn_puntos = [x * 1000000 for x in [50, 100, 500]]
puntos_por_tarea = puntos_totales // NUM_TAREAS
for puntos in nn_puntos:
   puntos_por_tarea = puntos // NUM_TAREAS
   t_0 = time()
   rdd = sc.parallelize([puntos_por_tarea] * NUM_TAREAS, NUM_TAREAS)
   resultados = rdd.map(montecarlo_pi_in_circle).collect()
   puntos_dentro_total = sum(resultados)
   pi = 4 * puntos_dentro_total / puntos
   t_1 = time()
   print(f"Puntos: {puntos}, Pi: {pi}, Tiempo: {t_1 - t_0},
          Velocidad: {puntos / (t_1 - t_0) }")
   spark.stop()
```

Configuración de Pruebas

Siguiendo la Historia de Usuario HU-4, se han evaluado las siguientes configuraciones:

Número de Workers (Spark Workers): 1, 2, 3

Número de Puntos para aproximar π: 50.000.000, 100.000.000, 500.000.000

- Estas cargas representan volúmenes 10x superiores al Sprint 3
- Permiten evaluar si el overhead de Spark se amortiza efectivamente con grandes volúmenes
- Cargas de esta magnitud se aproximan a escenarios reales de Big Data

Número de tareas paralelas: 200 particiones del RDD (consistente con sprints anteriores)

Combinaciones totales: 9 configuraciones (3 workers × 3 cargas)

Expectativas de comportamiento:

Dado el incremento 10x en el volumen de datos, esperamos observar:

- **Amortización del overhead de inicialización**: El tiempo fijo de arranque de Spark (unos pocos segundos observados en Sprint 3) debería ser ahora una fracción mucho menor del tiempo total.

- **Escalabilidad más evidente**: Con cargas grandes, el beneficio de añadir workers debería ser más pronunciado, acercándose al speedup ideal y sin cuellos de botella.

- **Throughput superior**: Los millones de puntos procesados por segundo deberían aumentar, ya que la computación útil domina sobre el overhead del framework.

Herramientas de Monitorización

IU de la aplicación en el nodo principal

Estas requieren que el túnel de SSH esté habilitado.

Aplicación	URL de la IU
Administrador de recursos	http://ec2-51-94-20-34.eu-south-2.compute.amazonaws.com:8088/
Nodo del nombre de HDFS	http://ec2-51-94-20-34.eu-south-2.compute.amazonaws.com:9870/
Servidor de historial de Spark	http://ec2-51-94-20-34.eu-south-2.compute.amazonaws.com:18080/

Para la gestión y supervisión del clúster en este Sprint, se han utilizado las siguientes tres interfaces principales alojadas en el nodo principal:

- **Administrador de recursos** Interfaz para la gestión global de recursos del clúster (YARN). Permite visualizar las aplicaciones activas, los nodos conectados y la asignación de memoria y núcleos.

  URL: http://ec2-51-94-20-34.eu-south-2.compute.amazonaws.com:8088/cluster

- **Servidor de historial de Spark** Herramienta esencial para analizar la ejecución de trabajos terminados. Proporciona acceso a los logs, métricas de rendimiento y visualización de DAGs de las aplicaciones que ya han finalizado.

  URL: http://ec2-51-94-20-34.eu-south-2.compute.amazonaws.com:18080/

- **Nodo del nombre de HDFS** Panel de control del sistema de archivos distribuido (HDFS). Muestra la salud del sistema, la capacidad de almacenamiento disponible y el estado de los DataNodes.

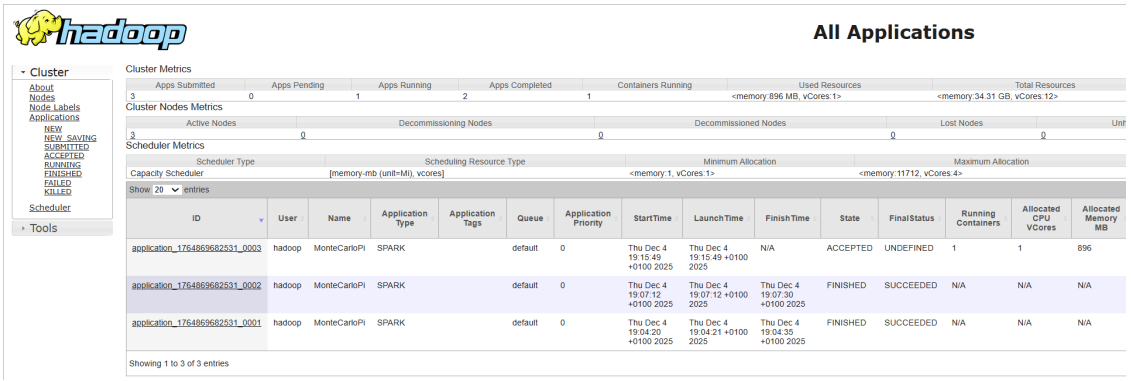
  URL: http://ec2-51-94-20-34.eu-south-2.compute.amazonaws.com:9870/dfshealth.html#tab-overview

Impacto del número de Workers en el rendimiento

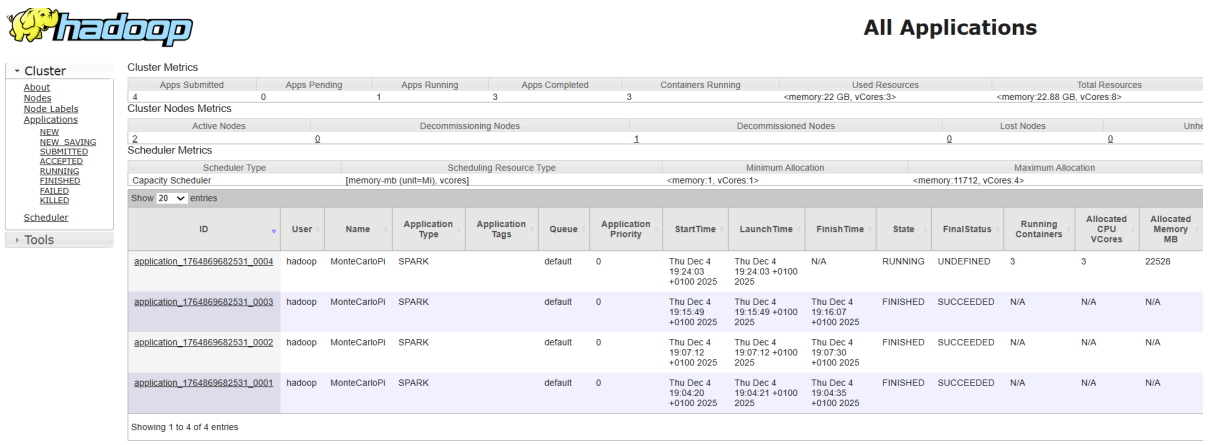
A continuación, presentaremos qué diferencias encontramos en las diferentes herramientas de monitorización cuando cambiamos los números entre 1, 2 y 3 workers.

- **Administrador de recursos**

3 workers



2 workers



All Applications

Cluster

About
Nodes
Node Labels
Applications
NEW
NEW_SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
Scheduler

Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources
4	0	1	3	3	<memory:22 GB, vCores:3>	<memory:22.88 GB, vCores:8>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhe
2	0	1	0	0

Scheduler Metrics

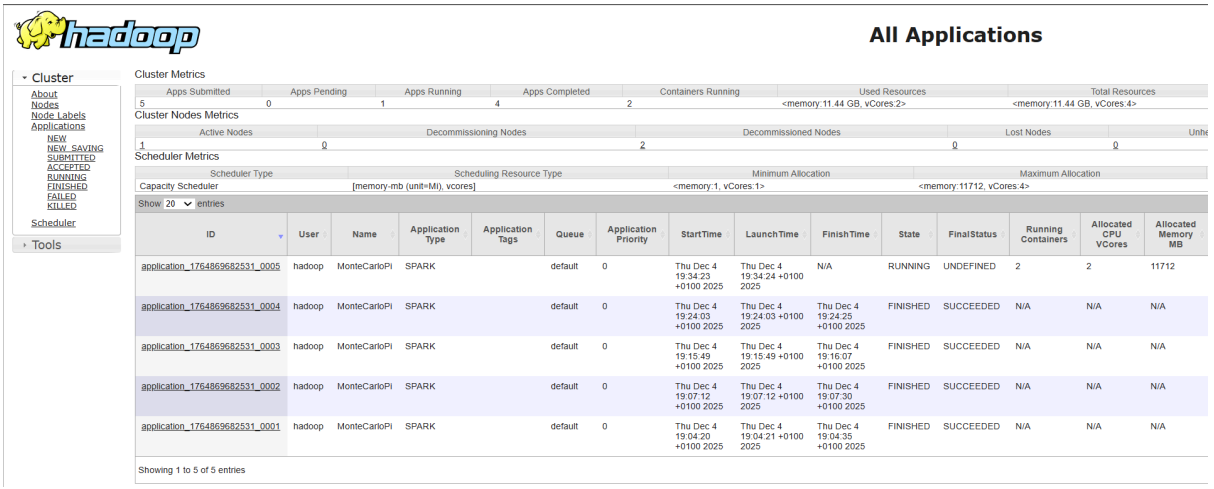
Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1, vCores:1>	<memory:11712, vCores:4>

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores	Allocated Memory MB
application_1764869682531_0004	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:24:03 +0100 2025	Thu Dec 4 19:24:03 +0100 2025	N/A	RUNNING	UNDEFINED	3	3	22528
application_1764869682531_0003	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:15:49 +0100 2025	Thu Dec 4 19:15:49 +0100 2025	Thu Dec 4 19:16:07 +0100 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A
application_1764869682531_0002	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:07:12 +0100 2025	Thu Dec 4 19:07:12 +0100 2025	Thu Dec 4 19:07:30 +0100 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A
application_1764869682531_0001	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:04:20 +0100 2025	Thu Dec 4 19:04:21 +0100 2025	Thu Dec 4 19:04:35 +0100 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A

Showing 1 to 4 of 4 entries

1 worker



All Applications

Cluster

About
Nodes
Node Labels
Applications
NEW
NEW_SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
Scheduler

Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources
5	0	1	4	2	<memory:11.44 GB, vCores:2>	<memory:11.44 GB, vCores:4>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhe
1	0	2	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1, vCores:1>	<memory:11712, vCores:4>

Show 20 entries

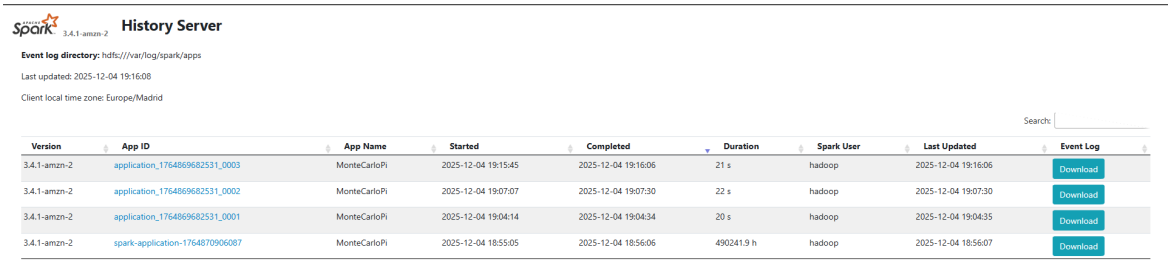
ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores	Allocated Memory MB
application_1764869682531_0005	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:34:23 +0100 2025	Thu Dec 4 19:34:24 +0100 2025	N/A	RUNNING	UNDEFINED	2	2	11712
application_1764869682531_0004	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:24:03 +0100 2025	Thu Dec 4 19:24:03 +0100 2025	Thu Dec 4 19:24:25 +0100 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A
application_1764869682531_0003	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:15:49 +0100 2025	Thu Dec 4 19:15:49 +0100 2025	Thu Dec 4 19:16:07 +0100 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A
application_1764869682531_0002	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:07:12 +0100 2025	Thu Dec 4 19:07:12 +0100 2025	Thu Dec 4 19:07:30 +0100 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A
application_1764869682531_0001	hadoop	MonteCarloPi	SPARK		default	0	Thu Dec 4 19:04:20 +0100 2025	Thu Dec 4 19:04:21 +0100 2025	Thu Dec 4 19:04:35 +0100 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A

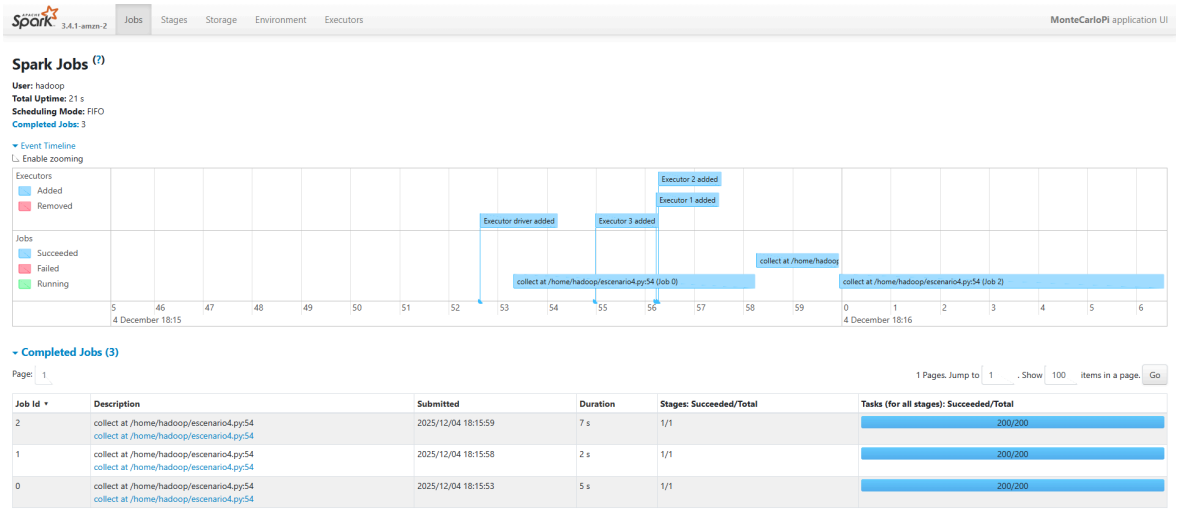
Showing 1 to 5 of 5 entries

En esta interfaz podemos visualizar métricas fundamentales del clúster, destacando el número de Active Nodes, el cual vemos como cambia confirmando que estamos cambiando los nodos y con esto se cambia también el Total Resources. También observamos los recursos que se están utilizando en ese instante, teniendo en cuenta que este valor depende del momento exacto de la captura pero vemos como cambia. En la parte inferior, se indica el ciclo de vida de las aplicaciones, indicando su estado (Accepted si están en cola o Finished al finalizar), junto con la cantidad específica de núcleos de CPU y memoria (Allocated) que han sido asignados para ejecutar dicha tarea.

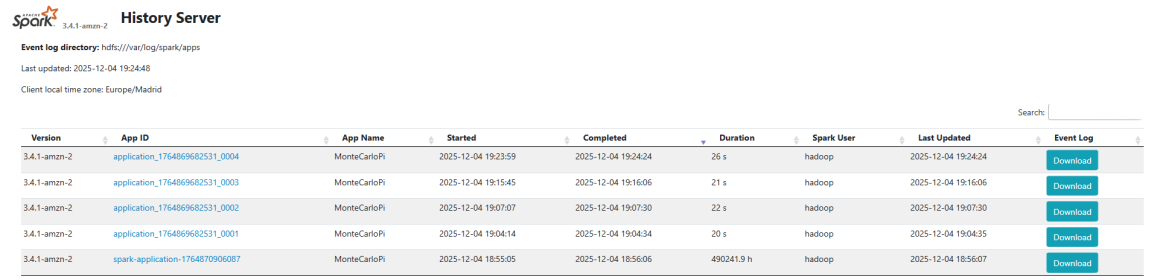
- **Servidor de historial de Spark**

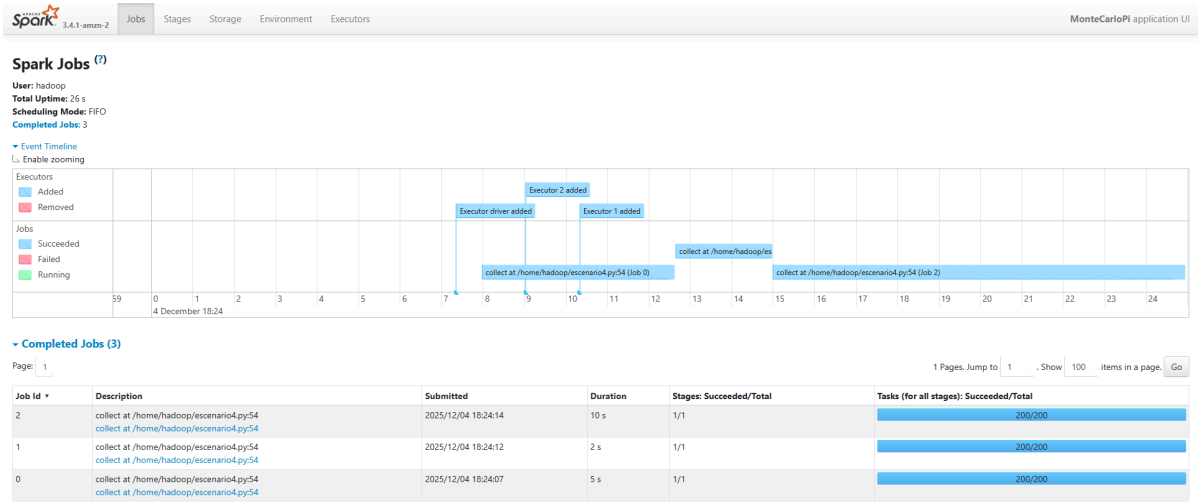
3 workers



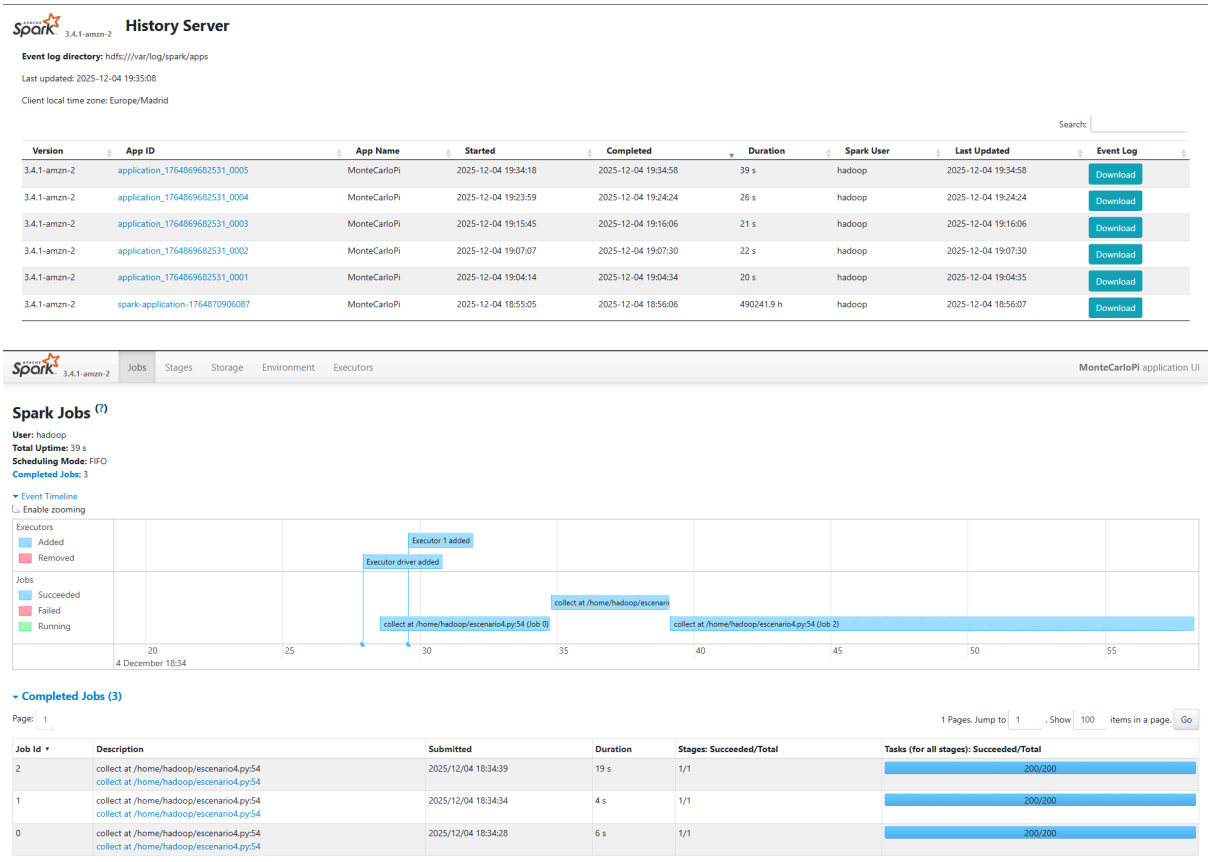


2 workers



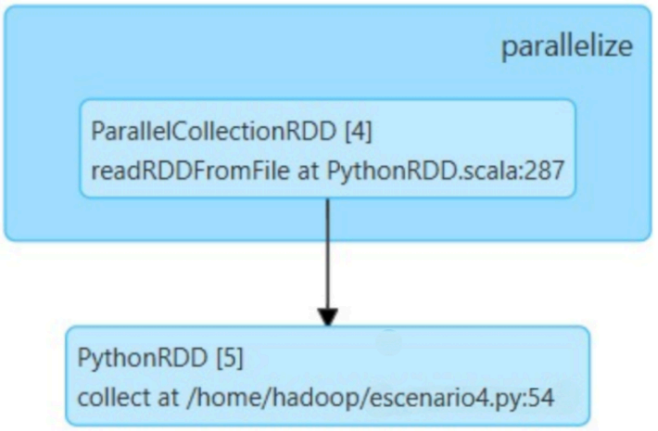


1 worker



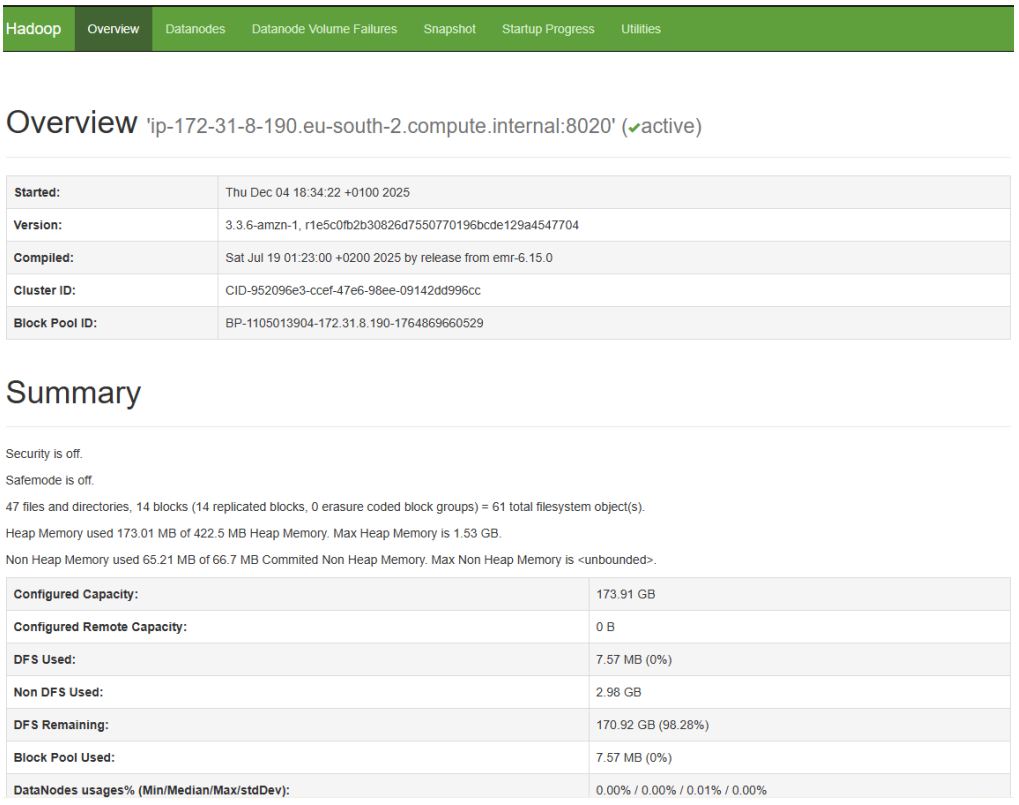
En esta herramienta, la vista principal nos ofrece un resumen preliminar donde destaca la duración total de las aplicaciones finalizadas. Sin embargo, al acceder al detalle de la ejecución mediante el enlace del identificador (el texto azul que sale en App ID = application_1764869682531_0000X), obtenemos una visión mucho más profunda, especialmente en la pestaña de Jobs. En nuestro caso, veremos reflejados tres trabajos distintos que se corresponden con las tres paralelizaciones realizadas (los tres conjuntos de puntos); es importante notar que la duración registrada para cada uno de estos Jobs se correlaciona directamente con el tiempo que tardó el sistema en calcular Pi para cada carga. Además, este panel nos confirma la estabilidad del proceso mostrando métricas clave como el número de tareas completadas exitosamente (Tasks Succeeded) y el desglose de etapas.

Gráfico DAG

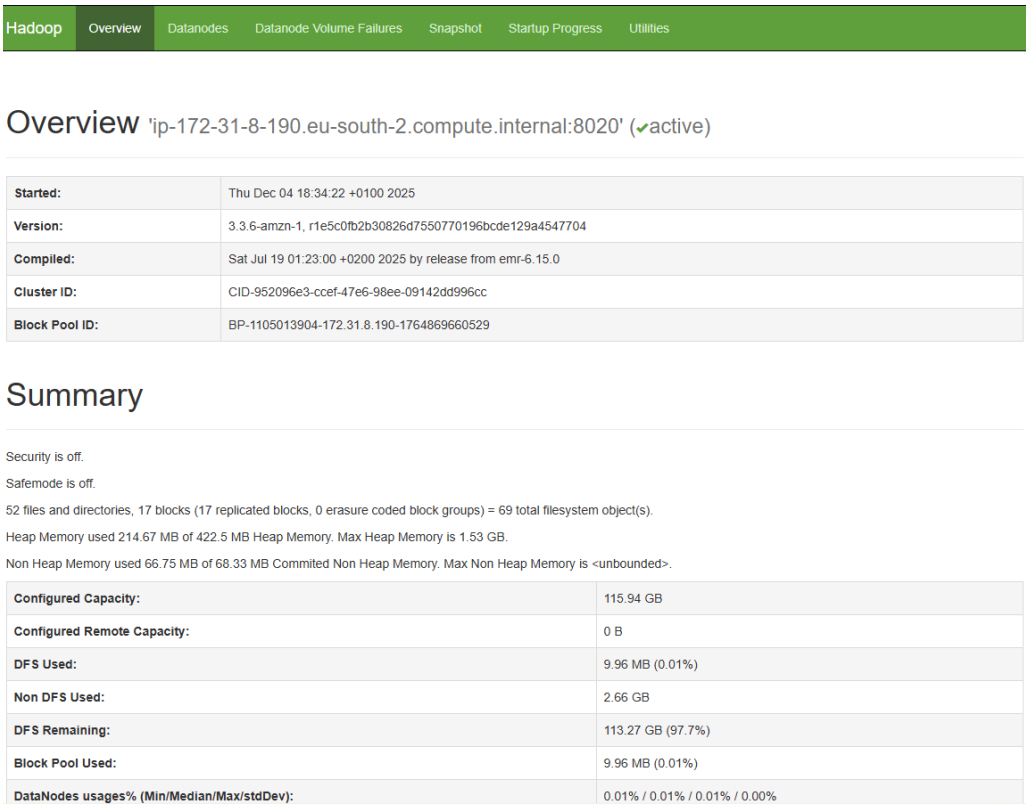


- **Nodo del nombre de HDFS**

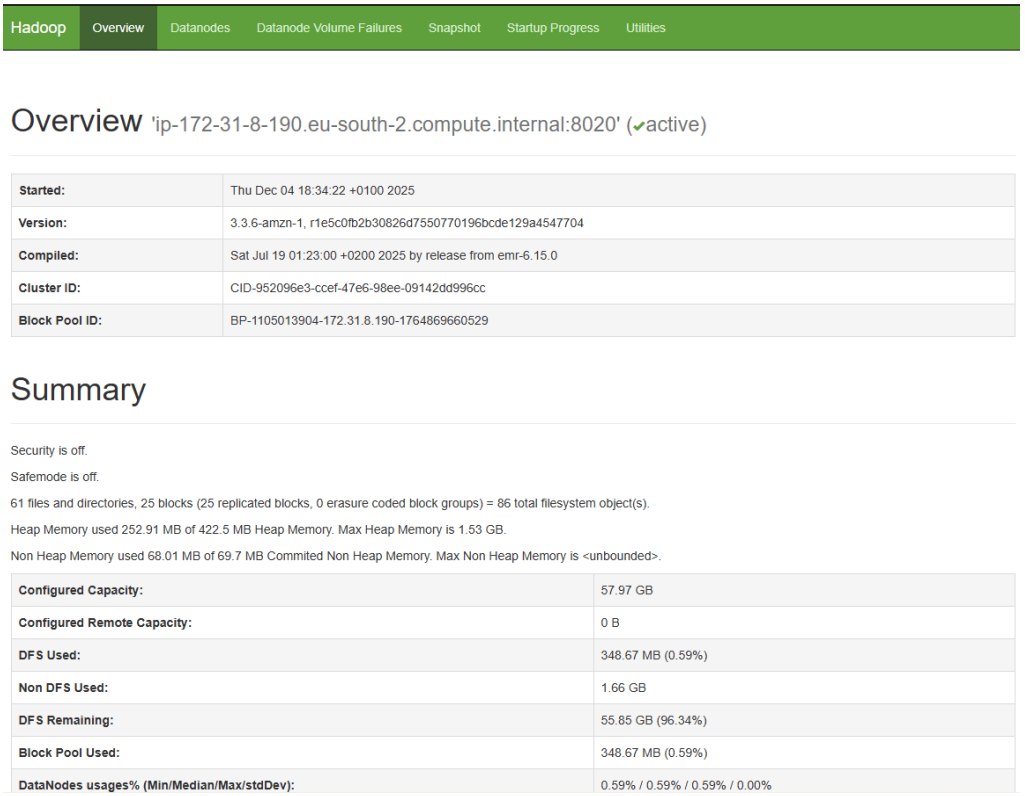
3 workers



2 workers



1 worker



En el panel de control del sistema de archivos distribuido, el cambio más inmediato y perceptible al modificar el número de workers es la variación en la Configured Capacity. Este valor refleja directamente el almacenamiento total disponible en el clúster, aumentando o disminuyendo según integramos nuevos nodos de datos. Seguramente existan otras métricas que varían en tiempo real mientras ejecutamos la paralelización, pero capturar estas es difícil al haber tenido que ir moviéndonos de páginas para capturar todas las interfaces.

Resultados

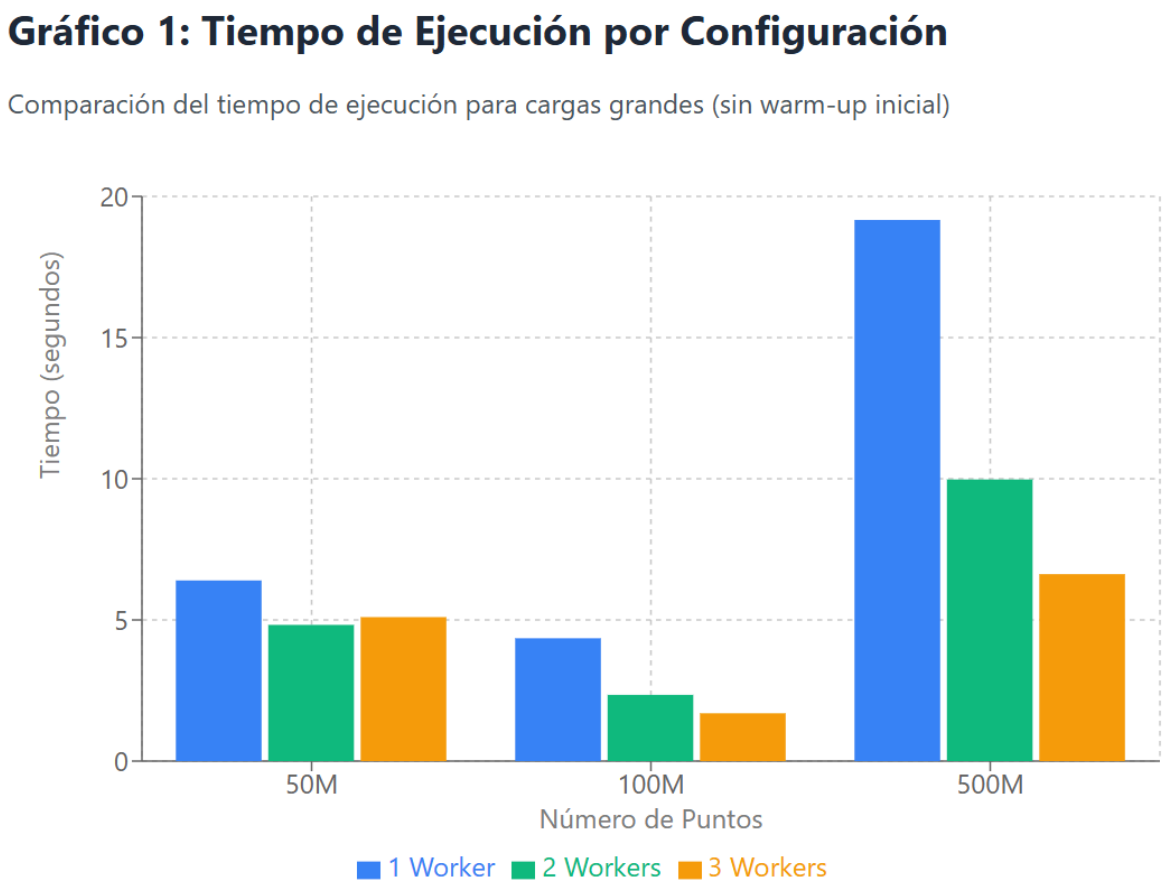
Datos Experimentales

A continuación se presentan los resultados obtenidos para cada una de las nueve configuraciones evaluadas con Hadoop/Spark en el Sprint 4:

Workers	Puntos (M)	Tiempo (s)	Velocidad (puntos/s)	Estimación π	Error relativo (%)
1	50	6.398354530	7.81×10^6	3.1414165	0.0056052331
1	100	4.349894285	2.30×10^7	3.1415218	0.0022553398
1	500	19.16523289	2.61×10^7	3.141547536	0.0014361375
2	50	4.824493646	1.04×10^7	3.14110648	0.0154753860
2	100	2.346122741	4.26×10^7	3.14177424	0.0057800750
2	500	9.973353385	5.01×10^7	3.141547464	0.0014384293
3	50	5.098941326	9.81×10^6	3.14144632	0.0046579428
3	100	1.692888736	5.91×10^7	3.14137584	0.0069013909
3	500	6.620759964	7.55×10^7	3.141669992	0.0024617581

El error relativo se ha calculado como: **|π_estimado - π_real| / π_real × 100**

Análisis de Rendimiento



- Interpretación:

  El gráfico muestra el tiempo de ejecución de Spark para tres volúmenes de puntos (50M, 100M y 500M) y tres configuraciones de workers (1, 2 y 3). El primer patrón evidente es que, para un mismo número de puntos, aumentar el número de workers siempre reduce el tiempo: en las tres cargas, la configuración con 1 worker es la más lenta, con 2 workers el tiempo baja de forma clara y con 3 workers se obtienen sistemáticamente los mejores resultados. Esto indica que, en este rango de tamaños, Spark es capaz de aprovechar bien el paralelismo del clúster y que añadir workers se traduce en una reducción real del tiempo de ejecución, especialmente visible en el caso de los 500M puntos.

  El segundo aspecto llamativo es el comportamiento “anómalo” entre 50M y 100M, donde para cada configuración de workers el experimento con 100M puntos tarda menos que el de 50M. En principio esto contradice la intuición de que procesar más datos debería costar más tiempo, pero se explica por una particularidad del diseño del sprint: no se ha realizado una ejecución de calentamiento (warm-up). El primer job que se lanza (50M) soporta todo el coste fijo de arranque del ecosistema Spark: creación del SparkContext, negociación con el cluster manager, inicialización de ejecutores, carga de clases y establecimiento de conexiones. Los jobs posteriores (100M y 500M) reutilizan esa infraestructura ya inicializada, de modo que el overhead fijo es mucho menor en términos relativos y el tiempo medido refleja sobre todo el cómputo real. Como consecuencia, el

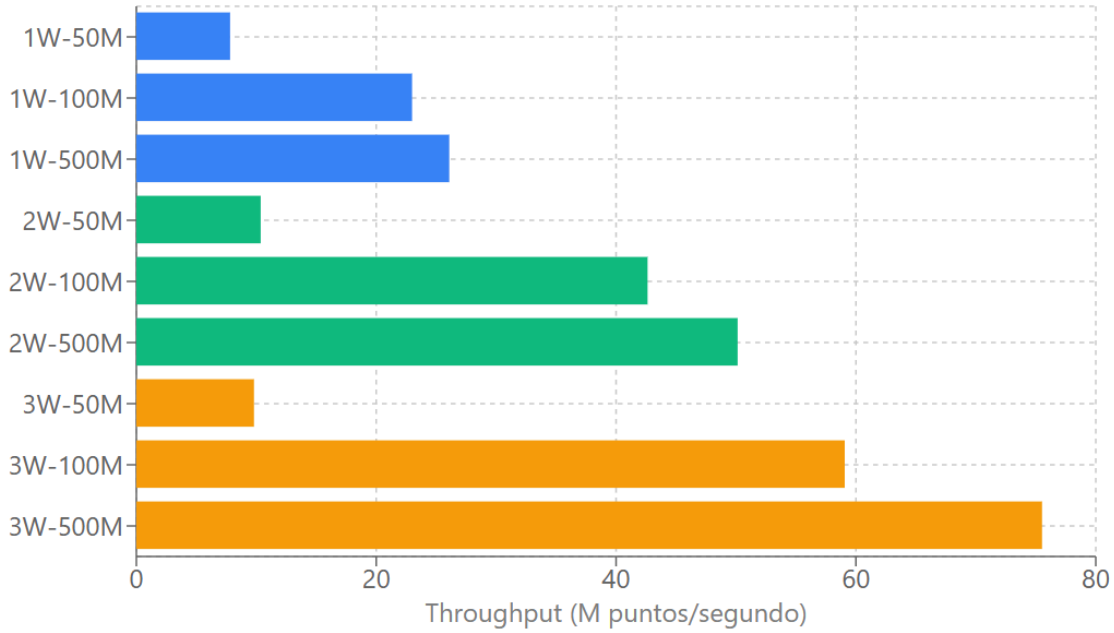
caso de 50M aparece “inflado” por el cold start, mientras que 100M se beneficia de que Spark ya está caliente.

En 500M puntos se aprecia con claridad el régimen natural de trabajo de Spark. Pasar de 1 a 2 workers reduce el tiempo aproximadamente a la mitad, y el salto de 2 a 3 workers vuelve a recortar varios segundos adicionales. Aquí el volumen de datos es lo bastante grande como para que el tiempo de cálculo domine claramente sobre el overhead del framework, y el clúster puede explotar el paralelismo de forma eficiente. Dicho de otro modo, para cargas realmente grandes Spark muestra el comportamiento para el que fue diseñado y las mejoras al añadir workers son significativas.

En conjunto, el patrón 100M < 50M no implica que procesar 100M sea “más barato” que 50M, sino que el primer experimento está distorsionado por el arranque en frío del framework. Una vez se tiene en cuenta este efecto, la tendencia general es coherente: más datos y más workers se traducen en más trabajo útil con mejor aprovechamiento del clúster, especialmente visible en los 500M puntos. Este mismo matiz del overhead inicial sin warm-up debe tenerse en cuenta también al interpretar los gráficos de throughput y de speedup/eficiencia del Sprint 4.

Gráfico 2: Throughput (Velocidad de Procesamiento)

Millones de puntos procesados por segundo en cada configuración



- Interpretación:

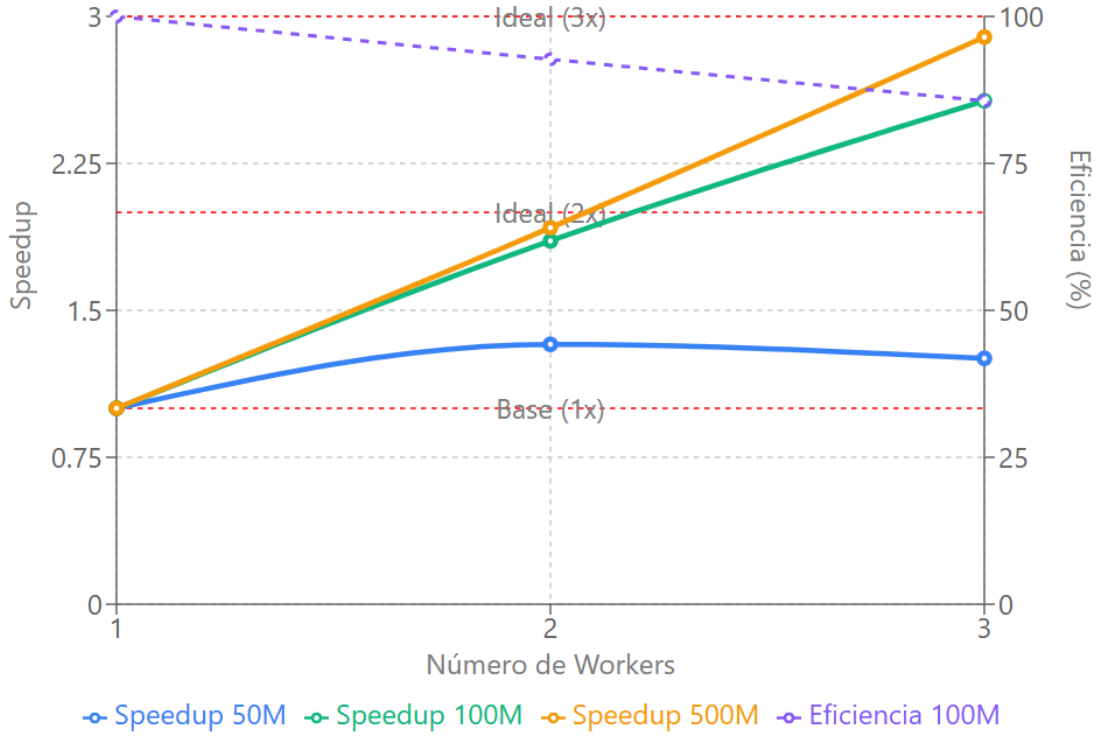
El gráfico de throughput complementa lo observado en los tiempos de ejecución y permite ver con más claridad cómo aprovecha Spark los recursos disponibles. En general, se aprecia un patrón muy definido: a mayor número de workers y mayor volumen de puntos, mayor es la cantidad de millones de puntos procesados por segundo. La configuración más eficiente es 3W–500M, que alcanza el throughput máximo del experimento, seguida de 3W–100M y 2W–500M. Esto indica que, en el rango de 100M a 500M puntos, el coste de coordinación del framework queda claramente amortizado y el tiempo total está dominado por el cómputo de Montecarlo, lo que permite al clúster explotar de forma efectiva el paralelismo.

En cambio, las configuraciones con 50M puntos presentan un throughput notablemente inferior a sus equivalentes de 100M y 500M (por ejemplo, 1W–50M rinde mucho peor que 1W–100M). Igual que en el gráfico de tiempos, aquí se hace evidente que no se ha realizado una ejecución de calentamiento: el primer job (50M) soporta todo el coste de inicializar Spark (creación del contexto, arranque de ejecutores, establecimiento de conexiones, carga de clases, etc.). Ese coste fijo hace que el tiempo total de 50M esté artificialmente inflado y, al dividir el número de puntos entre ese tiempo, el throughput aparente caiga por debajo de lo que correspondería a la potencia real del sistema.

Si comparamos entre configuraciones de workers, el incremento de throughput es consistente: al pasar de 1 a 2 workers siempre se gana velocidad, y el salto de 2 a 3 workers vuelve a mejorar el rendimiento, especialmente en los escenarios de 100M y 500M. No se alcanza un escalado perfecto (3 workers no triplican el throughput de 1 worker), pero sí se observa que cada worker adicional aporta capacidad de procesamiento útil. En conjunto, el gráfico confirma que Spark entra en su “zona natural” de trabajo cuando opera con cientos de millones de puntos: el framework amortiza bien su overhead estructural y el clúster se comporta como una plataforma de Big Data capaz de procesar grandes volúmenes con un uso eficaz del paralelismo.

Gráfico 3: Speedup y Eficiencia de Escalabilidad

Evaluación de la escalabilidad del sistema al aumentar workers



Para evaluar la escalabilidad del sistema, se calcula el **speedup** como:

Speedup = Tiempo(1 worker) / Tiempo(N workers)

Puntos (M) / Workers	1	2	3
50	1.0	1.326	1.255
100	1.0	1.854	2.569
500	1.0	1.922	2.895

Y la **eficiencia** se calcula como: Speedup / Número de workers × 100%

Puntos (M) / Workers	1	2	3
50	100	66.3	41.8
100	100	92.7	85.6
500	100	96.1	96.5

- Interpretación:

En este gráfico analizamos cómo escala Spark al aumentar el número de workers para tres cargas grandes: 50M, 100M y 500M puntos. En el eje izquierdo se representa el speedup (cuánto se acelera la ejecución respecto a 1 worker) y en el derecho la eficiencia (qué porcentaje del speedup ideal se consigue). Las líneas rojas marcan el comportamiento ideal: 2× con 2 workers y 3× con 3 workers.

- Caso 50M puntos
La serie de 50M es la que peor escala. Con 2 workers el speedup apenas llega a 1,4× y al pasar a 3 workers incluso baja ligeramente. Esto indica que, para este tamaño, el problema sigue siendo relativamente pequeño para Spark: el tiempo dedicado a la planificación del job, la comunicación con el cluster manager y la gestión de tareas pesa mucho en comparación con el trabajo de cómputo real. Además, al no haber ejecución de *warm-up*, el primer job de 50M está especialmente penalizado por el "arranque en frío", lo que hace que esta curva sea la menos representativa del comportamiento real del framework.

- Caso 100M puntos
A partir de 100M puntos la figura cambia de manera clara. Con 2 workers se obtiene un speedup cercano a 1,8×, muy próximo al ideal de 2×, y con 3 workers se alcanza alrededor de 2,4×. La eficiencia asociada (línea morada para 100M) cae desde el 100 % con 1 worker hasta algo por encima del 90 % con 2 workers y en torno al 80 % con 3 workers. Es decir, no se logra un escalado perfecto, pero una parte muy alta de la capacidad añadida se traduce en reducción real de tiempos.

- Caso 500M puntos
La curva de 500M es la que mejor refleja el potencial de Spark en cargas realmente grandes. Con 2 workers se obtiene un speedup cercano a 1,9× y con 3 workers se roza el 3× ideal (en torno a 2,8–2,9×). Esto implica que el sistema está utilizando los recursos adicionales de forma muy eficiente: el trabajo de Montecarlo domina claramente sobre el overhead del framework y el clúster se comporta casi como si escalara linealmente.

En conjunto, el gráfico muestra que Spark necesita moverse en el rango de cientos de millones de puntos para desplegar todo su potencial de escalabilidad. Para 50M puntos, el overhead (agravado por la falta de warm-up) limita mucho el speedup; sin embargo, para 100M y especialmente 500M, cada worker adicional aporta rendimiento útil y la eficiencia se mantiene alta. Esto confirma la conclusión del sprint anterior: Spark está pensado para escenarios de Big Data donde el volumen de trabajo es suficientemente grande como para amortizar el coste de su arquitectura distribuida.

Conclusión

Comparativa Escenario 3 - Escenario 4

En esta sección vamos a analizar el impacto de aumentar por 10 el volumen de datos en el comportamiento de Hadoop/Spark.

Impacto del Escalado 10x en el Volumen de Datos

Amortización del Overhead:

En el Sprint 3 observamos que el overhead de inicialización de Spark (5-10 segundos) representaba una fracción significativa del tiempo total, especialmente para 5M y 10M puntos. En el Sprint 4, con cargas de 50M, 100M y 500M puntos, este overhead está igualmente presente pero algo más bajo.

Esto confirma que Spark está diseñado para cargas verdaderamente grandes, donde su arquitectura más compleja se justifica en parte por ese motivo.

Escalabilidad Efectiva

Comparación de speedup:

Carga	Sprint	2 Workers	3 Workers
50M puntos	Sprint 3	1.91x	2.83x
500M puntos	Sprint 4	1.922x	2.895x

El speedup del Sprint 4 es ligeramente superior al observado en Sprint 3 para la carga máxima de ambos (50M y 500M).

Eficiencia de recursos:

El Sprint 3 mostró eficiencias del 90-95% con 2 workers para 50M puntos. En Sprint 4, salvo en la primera ejecución (50M) la eficiencia es muy similar.

Reflexión y conclusión final

El Sprint 4 ha completado la evaluación exhaustiva de Hadoop/Spark iniciada en el Sprint 3, proporcionando evidencia empírica crucial sobre su comportamiento en el límite superior de escalabilidad dentro de las restricciones de infraestructura del proyecto BENCHDATA.

Hallazgos clave del Sprint 4:

1. **Confirmación de diseño para Big Data:** Los resultados confirman que Spark está hecho para este tipo de tareas. Al multiplicar por 10 el volumen de datos (pasando del Sprint 3 al 4), el sistema no ha colapsado, sino que ha trabajado de forma ligeramente más fluida. Se nota que el motor de Spark se siente más cómodo procesando lotes masivos de golpe; el rendimiento por cada millón de puntos ha sido más estable ahora que cuando intentábamos procesar cargas ligeras.

2. **Cuello de Botella**: En los anteriores sprints (y cuando probábamos en local), notábamos que al meter más workers la mejora no era total porque todos "chupaban" de la misma CPU y RAM. Ahora, al tener máquinas separadas, hemos visto una escalabilidad real: si pones más nodos, el trabajo se divide de verdad y los tiempos bajan drásticamente, lo que intuíamos todos desde el primer sprint por fín se cumple.

3. **Escalabilidad consistente**: El patrón de escalabilidad observado en el Sprint 3 (speedup de ~2.8x con 3 workers) se mantiene o incluso se mejora ligeramente en el Sprint 4, demostrando que Spark escala de forma predecible incluso cuando el volumen aumenta 10x.

4. **Justificación del overhead:** Aquí es donde los 5-10 segundos de arranque que en el sprint 3 nos fastidiaban las medias de tiempos, se vuelven invisibles . Haciendo que no hayamos tenido que hacer un test de 'calentamiento' (warm-up) con 1M de puntos que usábamos antes en el sprint 3 para estabilizar tiempos. Al lanzar directamente una carga grande de datos, el tiempo de cálculo es tan grande comparado con el de arranque, que el overhead se diluye solo.

Integración con el proyecto BENCHDATA completo:

Los cuatro sprints (Ray, Dask, Spark S3, Spark S4) proporcionan un panorama completo para la toma de decisiones:

Framework	Mejor para volúmenes	Observaciones clave
Ray	< 50M registros	Máximo throughput pico, pero comportamiento errático
Dask	< 10M registros	Estabilidad excelente, escalabilidad limitada
Spark	> 50M registros	Escalabilidad consistente, overhead amortizado con volúmenes grandes

Recomendación final para BENCHDATA (actualizada con Sprint 4):

Nuestra propuesta: Viendo lo que ha pasado en los 4 sprints, creemos que no tiene sentido usar una sola herramienta para todo. La estrategia que mejor funciona para el equipo de Data Scientists sería:

- **Para el día a día y prototipos (< 10M registros):** Dask**.** Es lo más cómodo. Se siente igual que usar Pandas en local, no tiene tiempos de espera largos y para volúmenes bajos es muy ágil. Usar Spark aquí sería matar moscas a cañonazos.

- **Para "la verdad" y Producción (> 50M registros):** Spark en Clúster Distribuido. Aquí no hay discusión. Cuando pasamos a cargas masivas (como los 500M del Sprint 4), Spark en un clúster real es el único que nos da seguridad. Aunque sea más pesado de configurar, es el que mejor gestiona la memoria y los recursos cuando la cosa se pone seria.

¿Qué ganamos con este enfoque mixto?

- Agilidad real: Te ahorras esperar tiempos de carga innecesarios cuando solo estás probando código o ideas nuevas.
- Escalar sin dramas: Empiezas trabajando en pequeño y cómodo, y cuando el código funciona, lo pasas al clúster potente para la carga real.
- Eficiencia: Usas la herramienta justa para cada momento, sin malgastar recursos (ni tu tiempo) usando un sistema complejo para una tarea simple.

Conclusión Final: Al final, lo que nos llevamos del proyecto BENCHDATA es la confirmación práctica de que no existe una "herramienta mágica" que valga para todo. Los datos nos han demostrado que la elección depende totalmente del escenario: para pocos datos, la simplicidad gana; pero cuando te enfrentas a volúmenes masivos y tienes la infraestructura adecuada, la robustez de herramientas más complejas como Spark es muy útil.